



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Laboratorio S08

saga orquestada y circuit breaker

MATERIA

Sistemas Distribuidos

Alumno

Reyes Vázquez Fernando

PROFESOR

Oscar Manuel Ruiz Hurtado

FECHA DE ENTREGA | 17 DE OCTUBRE DE 2026

GRUPO 02 | SEMESTRE 2027-1

1. El problema

Primero levanté los servicios de Mexi Banco con Docker Compose y preparé dos cuentas para reproducir el problema. La cuenta origen inició con 1000 y la cuenta destino con 500. Consulté los saldos por el gateway para comprobar que ambos valores estuvieran correctos antes de hacer la transferencia.

```
Date: Sat, 17 Oct 2026 03:02:29 GMT

{"id":5,"clabe":"2222222222222222","titular":"Cuenta Destino S08","saldo":500}%
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % curl -i http://localhost:8080/api/cuentas/111
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 2c33cc6bb765
Content-Type: application/json
Date: Sat, 17 Oct 2026 03:02:52 GMT

{"id":4,"clabe":"1111111111111111","titular":"Cuenta Origen S08","saldo":1000.00}%
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % curl -i http://localhost:8080/api/cuentas/222
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 2c33cc6bb765
Content-Type: application/json
Date: Sat, 17 Oct 2026 03:03:01 GMT

{"id":5,"clabe":"2222222222222222","titular":"Cuenta Destino S08","saldo":500.00}%
```

Figura 1. *Creación y consulta de las cuentas de prueba: 1000 en origen y 500 en destino.*

Después inicié transferencia con una pausa de 20 segundos entre el cargo y el abono. Durante esa pausa envié una transferencia de 200 y detuve las tres réplicas de cuenta. De esta forma, el cargo alcanzó a confirmarse, pero el abono ya no pudo realizarse.

```
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia
[+] up 3/3
 ✔ Container mexi-banco-v06-discovery-1     Healthy                                                                    6.3s
 ✔ Container mexi-banco-v06-db-1            Healthy                                                                   11.3s
 ✔ Container mexi-banco-v06-transferencia-1 Started                                                                   10.3s
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % docker compose ps transferencia
NAME                           IMAGE                          COMMAND                  SERVICE         CREATED          STATUS                    PORTS
mexi-banco-v06-transferencia-1   mexi-banco-v06-transferencia   "java -jar app.jar"   transferencia   38 seconds ago   Up 27 seconds (healthy)   8080/tcp
```

Figura 2. *Configuración de la pausa de 20 segundos en el servicio transferencia.*

```
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % curl -i -X POST http://localhost:8080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"1111111111111111","claveDestino":"2222222222222222","monto":200}'
HTTP/1.1 504 Gateway Timeout
Content-Type: application/json
Content-Length: 140

{"timestamp":"2026-10-17T03:20:25.685+00:00","path":"/api/transferencias","status":504,"error":"Gateway Timeout","req
```

```
mexi-banco-s08 — -zsh — 202×25

Last login: Fri Oct 16 19:20:54 on ttys011
fernandoreyesvazquez@Air-de-Fernando ~ % cd ~/Documents/SD_Clase/mexi-banco-s08
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % pwd
/Users/fernandoreyesvazquez/Documents/SD_Clase/mexi-banco-s08
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % docker compose stop cuenta
[+] stop 3/3
 ✔ Container mexi-banco-v06-cuenta-3 Stopped
 ✔ Container mexi-banco-v06-cuenta-2 Stopped
 ✔ Container mexi-banco-v06-cuenta-1 Stopped
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % docker compose up -d cuenta
[+] up 5/5
 ✔ Container mexi-banco-v06-discovery-1 Healthy
 ✔ Container mexi-banco-v06-db-1        Healthy
 ✔ Container mexi-banco-v06-cuenta-1    Started
 ✔ Container mexi-banco-v06-cuenta-2    Started
 ✔ Container mexi-banco-v06-cuenta-3    Started
```

Figura 3. *Transferencia interrumpida y detención del servicio cuenta durante la espera.*

Cuando volví a levantar las cuentas, el origen quedó con 800 y el destino continuó en 500. La operación devolvió un timeout, pero lo importante fue que el saldo ya había cambiado en un servicio y no en el otro.

```
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % docker compose ps cuenta
NAME                      IMAGE                   COMMAND              SERVICE   CREATED
mexi-banco-v06-cuenta-1   mexi-banco-v06-cuenta   "java -jar app.jar"   cuenta    34 minutes ag
mexi-banco-v06-cuenta-2   mexi-banco-v06-cuenta   "java -jar app.jar"   cuenta    34 minutes ag
mexi-banco-v06-cuenta-3   mexi-banco-v06-cuenta   "java -jar app.jar"   cuenta    34 minutes ag
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % curl -i http://localhost:8080/api/cuentas
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 2c33cc6bb765
Content-Type: application/json
Date: Sat, 17 Oct 2026 03:26:03 GMT

{"id":4,"clabe":"111111111111111111","titular":"Cuenta Origen S08","saldo":800.00}%
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % curl -i http://localhost:8080/api/cuentas
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 059d71e57d79
Content-Type: application/json
Date: Sat, 17 Oct 2026 03:26:11 GMT

{"id":5,"clabe":"222222222222222222","titular":"Cuenta Destino S08","saldo":500.00}%
```

Figura 4. *Consulta de saldos después de la falla: 800 en origen y 500 en destino.*

Cuenta	Antes	Después
Origen	$1,000.00	$800.00
Destino	$500.00	$500.00
Total	$1,500.00	$1,300.00

Esto pasó porque el cargo y el abono son peticiones HTTP independientes. Cada servicio confirma su propia operación y, si algo falla entre los dos, no existe una transacción única que revierta todo automáticamente.

2. Mi saga

Para corregir este comportamiento adapté TransferenciaService como orquestador de una saga. Primero comprueba que las cuentas existan y registra la transferencia en INICIADA. Después realiza el cargo, registra su movimiento, intenta el abono, registra el segundo movimiento y termina en COMPLETADA. En cada paso importante se actualiza el estado para saber hasta dónde llegó.

La acción de compensación principal es devolver el mismo monto a la cuenta origen cuando sabemos que el cargo sí se hizo, pero el abono fue rechazado. Si la devolución tampoco se puede confirmar, la transferencia queda como COMPENSACION_PENDIENTE en vez de aparentar que todo salió bien.

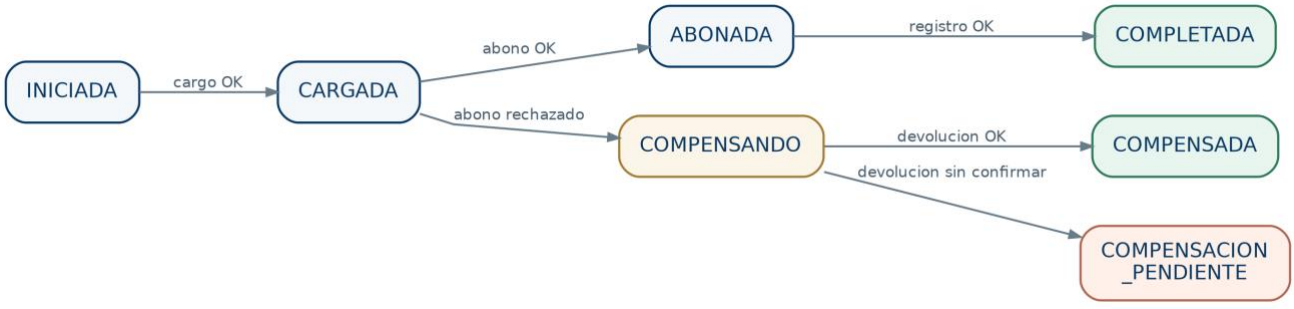


Figura 5. *Diagrama de estados con las rutas principales de una transferencia y su compensación.*

El código también contempla casos que requieren verificación, como CARGO_INCIERTO, ABONO_INCIERTO, REGISTRO_ABONO_PENDIENTE y REGISTRO_COMPENSACION_PENDIENTE, así como el rechazo FALLIDA.

Pasos y compensaciones

La consulta de cuentas solo verifica datos y no requiere compensación. El cargo a origen sí modifica el dinero; por eso, si falla una etapa posterior, la saga intenta abonarle nuevamente el monto. El movimiento del cargo permite dejar evidencia de esa salida y se verifica junto con la operación.

Si el abono a destino se confirma, no sería correcto regresarlo por una simple falla del historial o de las notificaciones: en esos casos se guardan estados pendientes de registro y se revisa el asiento que faltó. También distinguí el caso de un abono con resultado incierto, porque un timeout no demuestra por sí solo que el dinero no llegó.

Prueba de una transferencia normal

Para comprobar el recorrido completo creé dos cuentas nuevas, con saldos iniciales de 1000 y 500, y transferí 100. La petición respondió 201 Created y la transferencia 4 quedó con estado COMPLETADA.



Figura 6. *Creación de las cuentas y transferencia normal de 100, con respuesta COMPLETADA.*

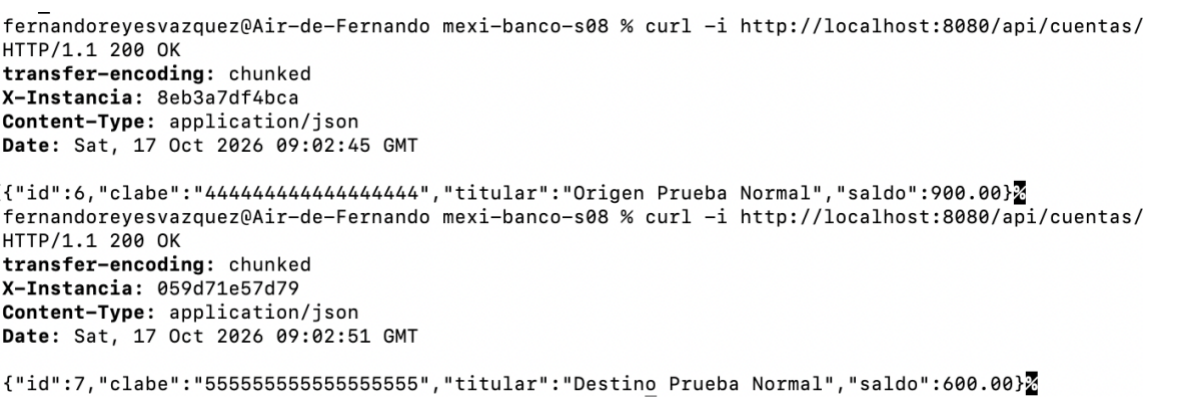


Figura 7. *Consulta de saldos tras la transferencia normal: 900 en origen y 600 en destino.*

Los saldos terminaron en 900 y 600. El total siguió siendo 1500, así que el dinero sí pasó correctamente de una cuenta a la otra.

3. El código clave

Estados que se guardan en la transferencia

Agregué un campo estado a la entidad Transferencia, guardado como texto con @Enumerated(EnumType.STRING). Así el valor queda registrado en PostgreSQL y podemos consultar si la operación terminó, se compensó o quedó pendiente.

Fragmento de Transferencia.java

```
@Enumerated(EnumType.STRING)
@Column(length = 40)
private EstadoTransferencia estado;


// En el constructor
this.estado = EstadoTransferencia.INICIADA;
```

En EstadoTransferencia.java definí estados para distinguir el recorrido normal y las fallas. Algunos son CARGADA, ABONADA, COMPLETADA, COMPENSANDO, COMPENSADA y COMPENSACION_PENDIENTE. Los estados de resultado incierto permiten identificar los casos que deben revisarse antes de volver a mover dinero.

El orquestador actualiza cada avance

Dentro de TransferenciaService.java agregué un método pequeño que cambia el estado, guarda el registro y escribe un mensaje en los logs. Esto me permitió observar el avance de la saga tanto desde PostgreSQL como desde la terminal.

Fragmento de TransferenciaService.java

```
private void guardarEstado(Transferencia t, EstadoTransferencia estado) {
    t.cambiarEstado(estado);
    transferencias.save(t);
    log.info("Transferencia {} -> {}", t.getId(), estado);
}
```

Falla simulada y compensación

Para poder probar la devolución sin apagar todas las cuentas, utilicé la variable FALLAR_ABONO_CLABE. Si la cuenta destino coincide, se simula un rechazo conocido antes de enviar el abono. La saga llama entonces al método de compensación.

Fragmentos de TransferenciaService.java

```
if (claveDestino.equals(clabeFallaAbono)) {
    throw new ErrorRemotoException("cuenta", 422,
        "Falla simulada del abono a la CLABE " + claveDestino);
}


// Si el abono fue rechazado y sabemos que no se ejecutó:
return compensar(t, claveOrigen, monto);
```

En compensar() se intenta abonar a la cuenta origen lo que ya se había cargado. Si se confirma la devolución y también queda registrado su movimiento, se marca COMPENSADA. Si el servicio no responde durante la devolución, el código guarda COMPENSACION_PENDIENTE para no asegurar algo que aún no se ha comprobado.

Fragmento central de compensar()

```
guardarEstado(t, EstadoTransferencia.COMPENSANDO);
try {
    origenDevuelto = cuentas.abonar(claveOrigen, monto);
} catch (RuntimeException errorDevolucion) {
    guardarEstado(t, EstadoTransferencia.COMPENSACION_PENDIENTE);
    return t;
}
```

Respuesta al cliente y protección de cuenta

También modifiqué TransferenciaController.java para que el cliente reciba un mensaje de acuerdo con el resultado. Una transferencia terminada regresa 201, una compensada devuelve 409 y un caso pendiente se responde como 202 para indicar que falta seguimiento. En CuentaCliente.java envolví las consultas, cargos y abonos con un circuit breaker de Resilience4j.

Fragmentos del controlador y de CuentaCliente.java

```
// TransferenciaController.java
case COMPLETADA -> HttpStatus.CREATED;
case COMPENSADA -> HttpStatus.CONFLICT;

// CuentaCliente.java
return circuito.executeSupplier(() ->
    http.get().uri("/cuentas/{clabe}", clabe)
        .retrieve().body(CuentaRemota.class));
```

Los fragmentos anteriores muestran las líneas principales de los archivos modificados. Se omitieron otras líneas para que el funcionamiento se entienda con mayor facilidad.

4. La compensación funcionando

Para probar la devolución configuré FALLAR_ABONO_CLABE con la CLABE de destino y dejé la pausa en cero. Envié una transferencia de 100 desde la cuenta origen, que tenía 800, hacia la cuenta destino, que tenía 500. Se provocó el rechazo del abono sin apagar cuenta.

```
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % FALLAR_ABONO_CLABE=2222222222222222 PAUSA_ENTRE_CARGO_Y_ABONO_MS=0 docker compose up -d transferencia
[+] up 3/3
 ✔ Container mexi-banco-v06-db-1            Healthy
 ✔ Container mexi-banco-v06-discovery-1     Healthy
 ✔ Container mexi-banco-v06-transferencia-1 Started
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % docker compose ps transferencia
NAME                            IMAGE                          COMMAND              SERVICE         CREATED          STATUS                    PORTS
mexi-banco-v06-transferencia-1   mexi-banco-v06-transferencia   "java -jar app.jar"   transferencia   43 seconds ago   Up 38 seconds (healthy)   8080/tcp
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % docker compose exec transferencia printenv FALLAR_ABONO_CLABE
2222222222222222
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % docker compose exec transferencia printenv PAUSA_ENTRE_CARGO_Y_ABONO_MS
0
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % curl -i -X POST http://localhost:8080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"1111111111111111","claveDestino":"2222222222222222","monto":100}'
HTTP/1.1 409 Conflict
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 17 Oct 2026 08:33:28 GMT

{"id":2,"claveOrigen":"1111111111111111","claveDestino":"2222222222222222","monto":100,"fecha":"2026-10-17T08:33:24.364475883Z","estado":"COMPENSADA","mensaje":"F
o al origen."}
```

Figura 8. *Variable de falla configurada y petición de 100 con respuesta 409 Conflict y estado COMPENSADA.*

La respuesta fue 409 Conflict con el estado COMPENSADA y un mensaje que indicó que el dinero se devolvió al origen. Después consulté las cuentas y ambas conservaron sus saldos originales: 800 y 500.

```
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % curl -i http://localhost:8080/api/cuentas/1111111111111111
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 2c33cc6bb765
Content-Type: application/json
Date: Sat, 17 Oct 2026 08:34:20 GMT

{"id":4,"clabe":"1111111111111111","titular":"Cuenta Origen S08","saldo":800.00}%
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % curl -i http://localhost:8080/api/cuentas/2222222222222222
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 2c33cc6bb765
Content-Type: application/json
Date: Sat, 17 Oct 2026 08:34:27 GMT

{"id":5,"clabe":"2222222222222222","titular":"Cuenta_Destino S08","saldo":500.00}%
```

Figura 9. *Saldos de origen y destino después de compensar: 800 y 500.*

También consulté la base transferencia de PostgreSQL. La operación quedó registrada con identificador 2, monto 100 y estado COMPENSADA. Así no dependimos únicamente del mensaje de la API para confirmar el resultado.

```
 id |   estado    | monto
----+-------------+--------
  2 | COMPENSADA  | 100.00
  1 |             | 200.00
(2 rows)
```

Figura 10. *Estado COMPENSADA guardado en PostgreSQL para la transferencia 2.*

5. Cuando la compensación también falla

Después repetí una transferencia con la pausa de 20 segundos, pero ahora apagué cuenta durante esa espera. Al intentar hacer el abono de destino, la comunicación falló; la saga quiso regresar los 50 a la cuenta origen, pero tampoco pudo confirmar la devolución. El gateway devolvió 504 Gateway Timeout durante la prueba.

Al consultar PostgreSQL, la transferencia 3 apareció con monto 50 y estado COMPENSACION_PENDIENTE. En los logs también se observa que pasó de CARGADA a COMPENSANDO y después a COMPENSACION_PENDIENTE. Esto es mejor que dejarla simplemente como CARGADA, porque identifica que todavía hay un problema por resolver.

```
 id |          estado          | monto
----+--------------------------+--------
  3 | COMPENSACION_PENDIENTE   |  50.00
  2 | COMPENSADA               | 100.00
  1 |                          | 200.00
(3 rows)
```

Figura 11. *Transferencia 3 por 50 con estado COMPENSACION_PENDIENTE en la base de datos.*

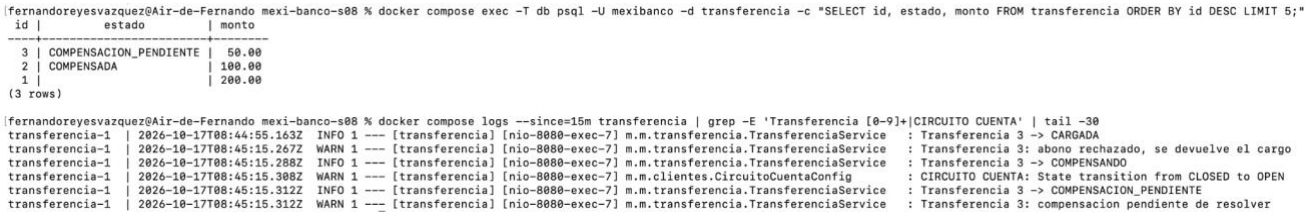


Figura 12. *Mensajes del orquestador durante la compensación pendiente y apertura del circuito.*

¿Cómo terminaría esta transferencia?

Primero revisaría los saldos y los movimientos para confirmar si el cargo y la devolución llegaron a ejecutarse. Si verifico que todavía falta devolver el dinero, realizaría una compensación controlada usando el identificador de la transferencia. Esa acción tendría que ser idempotente: repetirla con el mismo identificador no debería devolver 50 dos veces. Solo después de confirmar la devolución y su registro cambiaría el estado a COMPENSADA. En esta práctica dejamos registrada la operación pendiente; no implementamos un reintento automático.

6. El circuit breaker

El segundo problema que atendí fue evitar que transferencia siguiera llamando muchas veces a cuenta cuando sus réplicas estaban apagadas. Utilicé Resilience4j en CuentaCliente, de manera que las consultas, cargos y abonos compartieran el mismo circuito.

Los tres estados y la configuración

El circuito empieza CLOSED (cerrado), cuando deja pasar las llamadas. Al detectar suficientes fallas pasa a OPEN (abierto) y rechaza de inmediato nuevas solicitudes. Después de esperar, cambia a HALF_OPEN (semiabierto), permite una llamada de prueba y, si esta sale bien, regresa a CLOSED.

Fragmento de CircuitoCuentaConfig.java

```
CircuitBreakerConfig.custom()
    .slidingWindowSize(2)
    .minimumNumberOfCalls(2)
    .failureRateThreshold(50)
    .waitDurationInOpenState(Duration.ofSeconds(12))
    .permittedNumberOfCallsInHalfOpenState(1)
    .build();
```

Para dejarlo visible, detuve el servicio cuenta. Los logs mostraron el cambio de CLOSED a OPEN. Después lo recuperé, esperé el tiempo configurado y provoqué otra llamada sin mover dinero usando una CLABE inexistente. Aparecieron los cambios de OPEN a HALF_OPEN y de HALF_OPEN a CLOSED.

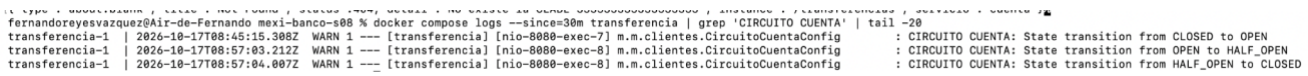


Figura 13. *Logs de Resilience4j con las transiciones CLOSED → OPEN → HALF_OPEN → CLOSED.*

```
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % curl -i http://localhost:8080/api/cuentas/333333333333333333
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: 059d71e57d79
Content-Type: application/problem+json
Date: Sat, 17 Oct 2026 08:56:42 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 333333333333333333","instance":"/cuentas/333333333333333333"}%
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % curl -i -X POST http://localhost:8080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"111111111111111111","claveDestino":"333333333333333333","monto":1}'
HTTP/1.1 404 Not Found
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Sat, 17 Oct 2026 08:57:04 GMT
```

Figura 14. *Prueba con una CLABE inexistente, que permitió revisar la recuperación sin hacer un cargo.*

Rechazo rápido mientras el circuito está abierto

También envié cinco solicitudes seguidas con cuenta apagada. La primera tardó 0.293 segundos y las siguientes respondieron con 503 Service Unavailable en tiempos mucho menores; la quinta tardó 0.008 segundos. En los logs volvió a observarse CLOSED to OPEN, lo que confirma que el servicio dejó de insistir en llamadas que iban a fallar.

```
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % docker compose stop cuenta
[+] stop 3/3
 ✔ Container mexi-banco-v06-cuenta-3 Stopped
 ✔ Container mexi-banco-v06-cuenta-1 Stopped
 ✔ Container mexi-banco-v06-cuenta-2 Stopped
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % for i in 1 2 3 4 5; do
  curl -s -o /dev/null \
    -w "Intento $i: HTTP %{http_code} - Tiempo: %{time_total}s\n" \
    -X POST http://localhost:8080/api/transferencias \
    -H "Content-Type: application/json" \
    -d '{"claveOrigen":"4444444444444444","claveDestino":"333333333333333333","monto":1}'
done
Intento 1: HTTP 500 - Tiempo: 0.293034s
Intento 2: HTTP 503 - Tiempo: 0.047167s
Intento 3: HTTP 503 - Tiempo: 0.007473s
Intento 4: HTTP 503 - Tiempo: 0.016510s
Intento 5: HTTP 503 - Tiempo: 0.008804s
fernandoreyesvazquez@Air-de-Fernando mexi-banco-s08 % docker compose logs --since=5m transferencia | grep 'CIRCUITO CUENTA' | tail -10
transferencia-1  | 2026-10-17T09:07:16.585Z  WARN 1 --- [transferencia] [nio-8080-exec-9] m.m.clientes.CircuitoCuentaConfig       : CIRCUITO CUENTA: State transition from CLOSED to OPEN
```

Figura 15. *Cinco intentos con el circuito abierto; las últimas respuestas 503 tardaron pocos milisegundos.*

Pruebas automatizadas

Para cerrar la revisión ejecuté las pruebas con Maven en un contenedor de Java 21. El resultado fue de 10 pruebas ejecutadas, sin fallas ni errores, y el proceso terminó con BUILD SUCCESS.

```
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  48.071 s
[INFO] Finished at: 2026-10-17T09:20:23Z
[INFO] ------------------------------------------------------------------------
```

Figura 16. *Resultado final de Maven: 10 pruebas, 0 fallas, 0 errores y BUILD SUCCESS.*

7. Lo que la saga no te da

Aunque la saga ayuda a recuperar el dinero cuando un paso falla, no ofrece el mismo aislamiento que una sola transacción en una base de datos. Entre el cargo y la compensación, otra persona que consulte el saldo de origen puede ver que ya se descontó el dinero, aunque la transferencia todavía no esté terminada. Por ejemplo, si el origen tenía 800 y se cargan 50, durante esos segundos podría mostrar 750; después de la compensación volvería a 800. La saga coordina y permite corregir el resultado, pero no esconde esos cambios intermedios a los demás servicios.